

Day 120 Part-2 — Final Beginner Python, Data Science and Machine Learning Mini Project

1. Day Number

Day 120 Part-2

You have reached the final beginner project in this 120 Part-2-day journey.

Today is about proving that you understand the **whole workflow**, not about building a production-grade ML system.

2. Topic Name

Simple End-to-End ML Model Tracked with MLflow and Exposed Through FastAPI

Your final mental model is:

```
data
  ↓
preprocessing
  ↓
model
  ↓
evaluation
  ↓
MLflow tracking
  ↓
prediction
  ↓
FastAPI
  ↓
JSON response
```

That is the core goal of Day 120 Part-2.

3. Connection to Your Day 1–120 Part-2 Journey

This project connects many skills you learned separately.

Earlier, you learned Python fundamentals such as:

```
variables
conditions
loops
functions
lists
dictionaries
```

```
files
exceptions
modules
OOP basics
```

Then you moved into developer and data tools:

```
Git
CSV
JSON
APIs
NumPy
Pandas
data cleaning
visualization
EDA
SQL
databases
```

Then mathematics:

```
algebra
vectors
matrices
statistics
probability
derivatives
gradients
```

Then machine learning:

```
features and labels
train/test split
overfitting
Linear Regression
Logistic Regression
KNN
Naive Bayes
Decision Tree
Random Forest
evaluation metrics
clustering
PCA
reinforcement learning ideas
```

Finally, you started learning the workflow around a model:

```
scikit-learn Pipeline
Kaggle-style workflow
MLflow
FastAPI
```

Today those pieces become one small application.

4. Revision Summary — Major Skills from Days 61–119

Rather than revising every day individually, group them into the major stages of your journey.

Days 61–70 — Developer Tools and Data Collection

You learned how Python projects interact with files and external data.

Important skills included:

```
Git
CSV
Excel
JSON
HTTP APIs
requests
basic debugging
try/except
```

A machine-learning project needs these skills because ML starts with **data**.

For example:

```
customers.csv
  ↓
Python
  ↓
Pandas
```

Days 71–77 — NumPy, Pandas and Data Cleaning

You learned:

```
NumPy arrays
Pandas Series
Pandas DataFrames
loading datasets
missing values
duplicates
data types
```

```
categorical encoding  
scaling
```

These skills form the beginning of today's project.

You should now understand:

```
raw data ≠ automatically model-ready data
```

Days 78–84 — Visualization, EDA and Data Preparation

You learned:

```
Matplotlib  
Seaborn  
EDA  
distributions  
relationships  
features  
labels  
train/test idea
```

Before training a model, you should understand what your dataset actually contains.

Days 85–91 — Databases and Mathematical Foundations

You studied:

```
SQL  
CRUD  
JOIN  
MySQL  
MongoDB  
Redis  
algebra  
vectors  
matrices
```

You do not need databases in today's tiny project, but you now understand that real applications often obtain data from systems rather than manually written Python lists.

Vectors and matrices also connect directly to ML because models usually process numerical feature matrices.

Days 92–98 — Statistics, Probability and Calculus

You learned concepts such as:

```
mean
median
variance
standard deviation
probability
distributions
Central Limit Theorem
derivatives
gradients
gradient descent idea
```

Those topics explain some of the mathematical reasoning behind models and metrics.

You do **not** need to manually implement those mathematics today.

Days 99–105 — Machine Learning Foundations

You learned:

```
machine learning lifecycle
features
labels
training data
validation/test data
overfitting
underfitting
bias
variance
feature engineering
Linear Regression
```

This is where individual data-analysis concepts started becoming ML workflows.

Days 106–112 — Supervised ML and Evaluation

You studied several models:

```
Logistic Regression
KNN
Naive Bayes
Decision Tree
Random Forest
```

And metrics such as:

```
accuracy
precision
recall
```

F1
MAE
MSE
RMSE

You also learned the basic idea of cross-validation.

For today's final project, we deliberately use only **one simple model**.

Days 113–116 — Unsupervised and Reinforcement Learning

You explored:

K-Means
DBSCAN
PCA
association learning
reinforcement learning
Q-Learning

Those concepts broadened your understanding of what machine learning can do.

Today's project returns to a simple **supervised model**, because it is easier to demonstrate the complete prediction API workflow.

Day 117 — Pipeline and Kaggle Workflow

You learned that preprocessing and a model can be connected using a scikit-learn Pipeline.

A Pipeline sequentially applies preprocessing transformers and ends with a predictor, which is exactly why it is useful for keeping training and prediction transformations together.

:chatgpt-content-reference{index="0"}

Conceptually:

```
raw features
  ↓
preprocessing
  ↓
model
  ↓
prediction
```

Days 118–119 — MLflow and FastAPI

You learned that MLflow can organize work into **experiments and runs**, with runs recording information such as parameters, metrics and artifacts. :chatgpt-content-reference{index="1"}

You also learned that FastAPI can receive structured request bodies and return responses. FastAPI commonly uses Pydantic models to describe and validate request-body data.

:chatgpt-content-reference{index="2"}

Today:

```
scikit-learn  
+  
MLflow  
+  
FastAPI
```

come together.

5. Important Topics

For Day 120 Part-2, concentrate on these topics:

```
project structure  
  
virtual environment  
requirements.txt  
  
Pandas  
  
basic data cleaning  
  
features  
label / target  
  
train/test split  
  
preprocessing  
  
scikit-learn Pipeline  
  
Logistic Regression  
  
model evaluation  
  
prediction  
  
MLflow experiment  
MLflow run  
parameters  
metrics  
artifacts/model  
  
FastAPI
```

endpoint

JSON request

JSON response

Do not try to demonstrate every algorithm you learned.

A small project that you fully understand is better than a large project assembled from copied code.

6. Foundational Notes

A. What are we actually building?

Imagine a tiny dataset that predicts whether a customer will purchase a product.

Possible columns:

age	income	visits	city	purchased
22	30000	2	Pune	0
31	55000	6	Bengaluru	1
45	72000	8	Mumbai	1
27	38000	3	Delhi	0

Here:

age
income
visits
city

are the **features**.

And:

purchased

is the **target/label**.

Because purchased is:

0 or 1

this is a **binary classification** problem.

A beginner-friendly model is:

Logistic Regression

B. Keep the dataset tiny

Your purpose is not:

Build the world's best purchasing model.

Your purpose is:

Demonstrate that I understand how data travels through an ML application.

Even something like:

```
50-200 simple rows  
4 features  
1 target
```

is enough conceptually.

C. Training and prediction are different stages

During training:

```
historical data  
  ↓  
fit pipeline  
  ↓  
learned model
```

Later:

```
new customer  
  ↓  
trained pipeline  
  ↓  
prediction
```

That distinction is extremely important.

D. Never fit preprocessing separately on future API input

Suppose training used:

```
StandardScaler  
OneHotEncoder  
LogisticRegression
```

Then API prediction should use that **same fitted preprocessing logic**.

Not:

```
API input
  ↓
create a brand-new scaler
  ↓
prediction
```

Instead:

```
API input
  ↓
already-fitted Pipeline
  ↓
prediction
```

This is one of the main reasons Pipeline is valuable.

E. MLflow is the experiment notebook

Think of MLflow as answering:

```
Which model did I train?

Which important settings did I use?

How well did it perform?

Which artifacts/model belonged to that run?
```

MLflow's current Tracking API supports logging parameters, metrics, tags and artifacts, and MLflow provides integrations for logging trained models. :chatgpt-content-reference{index="3"}

F. FastAPI does not train your model

FastAPI's responsibility is different.

It provides something like:

```
POST /predict
```

The client sends:

```
{
  "age": 32,
  "income": 60000,
```

```
"visits": 5,  
"city": "Bengaluru"  
}
```

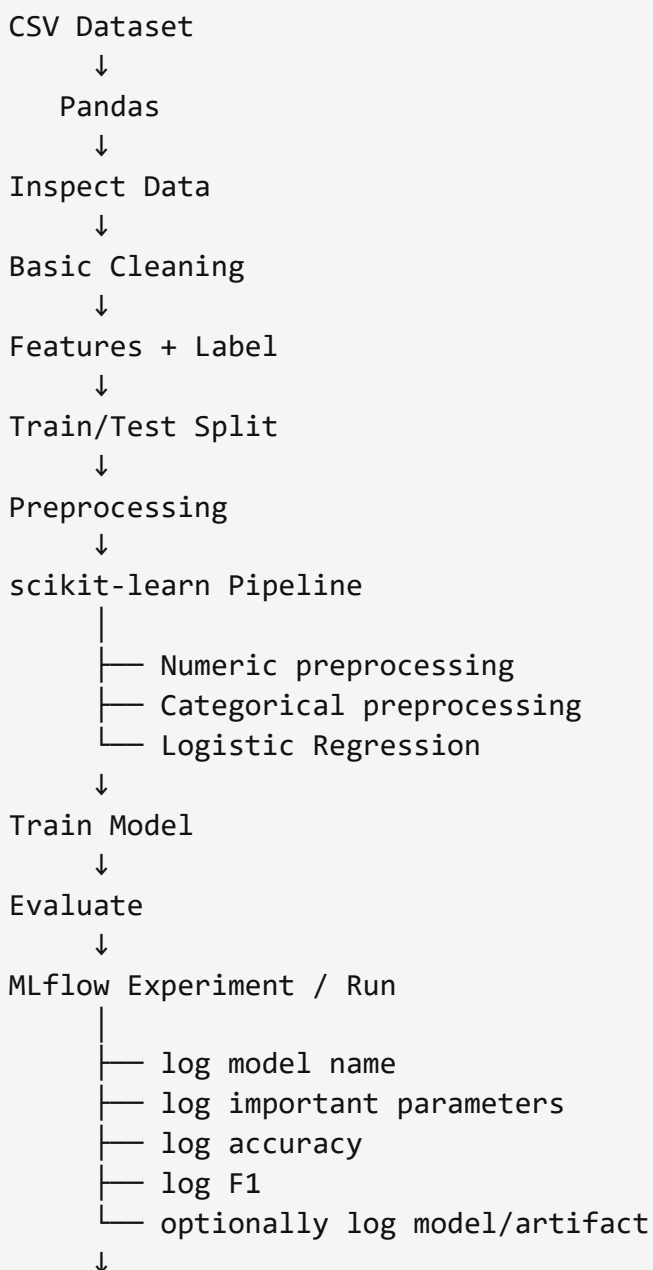
FastAPI passes those values to your prediction logic.

Then it returns something such as:

```
{  
  "prediction": 1  
}
```

7. Easy Architecture Diagram

Your complete beginner architecture looks like this:



```
Trained Pipeline
↓
Prediction Function
↓
FastAPI
↓
POST /predict
↓
JSON Request
↓
Prediction
↓
JSON Response
```

The critical idea is:

TRAINING SIDE

```
CSV
↓
Pandas
↓
Cleaning
↓
Pipeline
↓
Training
↓
Evaluation
↓
MLflow
```

APPLICATION SIDE

```
New JSON input
↓
FastAPI
↓
same trained Pipeline
↓
prediction
↓
JSON response
```

8. Final Problem Statement

Project: Tiny Customer Purchase Predictor

Build a **very small machine-learning prediction application**.

Your dataset might contain:

```
age
income
visits
city
purchased
```

where:

```
purchased = 0
```

means the customer did not purchase and:

```
purchased = 1
```

means they purchased.

Part A — Load the data

Use Pandas to load:

```
data/customers.csv
```

Think:

```
read CSV
store it in a DataFrame
```

Part B — Inspect the dataset

Check things such as:

```
first few rows
shape
column names
data types
missing values
target values
```

Do not immediately start training.

Ask first:

What kind of data am I dealing with?

Part C — Perform necessary basic cleaning

Only clean what actually needs cleaning.

For example:

```
missing age
duplicate row
incorrect numeric type
```

Do not create complicated cleaning logic just to make the project look advanced.

Part D — Separate features and target

Conceptually:

```
X = age + income + visits + city
y = purchased
```

Remember:

```
X → information used to predict
y → thing we want to predict
```

Part E — Train/test split

Split the data:

```
full dataset
  ↓
┌───┴───┐
│ training data │ testing data │
│   ↓         │   ↓         │
│ learn       │ evaluate      │
```

For example, conceptually:

```
80% train
20% test
```

Keep a fixed `random_state` for your simple experiment so that you can reproduce the same split.

Part F — Create simple preprocessing

You have two types of features.

Numeric:

```
age
income
visits
```

Categorical:

```
city
```

You could conceptually use:

```
numeric columns
    ↓
missing-value handling if needed
    ↓
StandardScaler

city
    ↓
missing-value handling if needed
    ↓
OneHotEncoder
```

Then combine them before the model.

Do not add preprocessing that your dataset does not need.

Part G — Keep preprocessing and model together

Create a scikit-learn Pipeline containing:

```
preprocessor
    ↓
Logistic Regression
```

Conceptually:

```
pipeline = Pipeline([
    ("preprocessing", ...),
```

```
("model", LogisticRegression(...))  
])
```

This is not the full solution—it simply shows the structure.

Part H — Train

Train using:

```
X_train  
y_train
```

Conceptually:

```
pipeline.fit(training features, training labels)
```

Because preprocessing is inside the Pipeline:

```
training data  
  ↓  
preprocessing  
  ↓  
model training
```

happens as one connected workflow.

Part I — Predict on test data

Use:

```
X_test
```

to produce predictions.

Conceptually:

```
X_test  
  ↓  
Pipeline  
  ↓  
preprocessing  
  ↓  
trained Logistic Regression  
  ↓  
predictions
```

Part J — Evaluate

Because this is classification, choose one or two appropriate metrics.

A simple choice:

```
accuracy  
F1
```

For example, your final output might conceptually say:

```
Accuracy: 0.83  
F1:      0.80
```

The exact values are not important for this learning project.

Understanding what produced them is more important.

Part K — Track the training run with MLflow

Create one experiment such as:

```
customer-purchase-project
```

Then create one main run:

```
logistic-regression-baseline
```

You do not need dozens of runs.

MLflow defines a run as one execution of your data-science code and an experiment as a grouping of related runs. :chatgpt-content-reference{index="4"}

Log important parameters

For example:

```
model_name = LogisticRegression  
test_size = 0.20  
random_state = 42
```

You could also record one meaningful model parameter such as:

```
C = 1.0
```

Conceptually:

```
log_param(...)
```

Remember:

Parameter = something about how the experiment was configured.

Log evaluation metrics

For example:

```
accuracy  
F1
```

Conceptually:

```
log_metric(...)
```

Remember:

Metric = a measured result.

So:

```
C = 1.0
```

is a parameter.

But:

```
F1 = 0.80
```

is a metric.

Optionally log the trained model

You may optionally log the Pipeline/model as part of the MLflow run.

Conceptually:

```
MLflow Run  
|  
├─ parameters  
├─ metrics  
└─ trained Pipeline/model
```

MLflow's official quickstart demonstrates manual parameter, metric and scikit-learn model logging within a run. [chatgpt-content-reference{index="5"}](#)

Or log one simple artifact

Instead of adding more complexity, you could create:

```
confusion_matrix.png
```

and store it as an artifact.

Then your run might look conceptually like:

```
Run: logistic-regression-baseline
```

Parameters

```
model = LogisticRegression
C = 1.0
test_size = 0.2
```

Metrics

```
accuracy = ...
F1 = ...
```

Artifacts

```
confusion_matrix.png
```

One artifact is enough.

Part L — Optional second run

Only after the main workflow works, you might describe one comparison.

For example:

```
Run 1
Logistic Regression
C = 1.0
```

```
Run 2
Logistic Regression
C = 0.5
```

Or perhaps:

```
Run 1
Logistic Regression
```

```
Run 2
Decision Tree
```

But this is optional.

Do **not** turn Day 120 Part-2 into:

```
50 runs
GridSearch
Optuna
huge tuning experiment
```

That would distract from the real goal.

Part M — Create a prediction function

Now imagine you already have your trained Pipeline.

Create a small function conceptually like:

```
predict_purchase(
    age,
    income,
    visits,
    city
)
```

Inside that function:

```
receive values
    ↓
create one-row structure
    ↓
send through trained Pipeline
    ↓
get prediction
    ↓
return prediction
```

The important part is:

```
prediction function
    ↓
uses the complete trained Pipeline
```

not just the final classifier.

Part N — Expose It Through FastAPI

Create one endpoint:

```
POST /predict
```

Why POST?

Because your client is sending structured data in the request body. FastAPI's documentation describes request bodies as client-supplied data and commonly uses Pydantic models to define them. :chatgpt-content-reference{index="6"}

Request

Conceptually:

```
{
  "age": 33,
  "income": 62000,
  "visits": 5,
  "city": "Bengaluru"
}
```

Flow

```
POST /predict
  ↓
FastAPI
  ↓
validate input
  ↓
prediction function
  ↓
trained Pipeline
  ↓
prediction
```

Response

For example:

```
{
  "prediction": 1
}
```

Or slightly more readable:

```
{
  "prediction": "purchase"
}
```

Keep it simple.

9. Concepts Used

This one project brings together:

Python modules
functions
exceptions

CSV

Pandas
DataFrame
missing values

features
label

classification

train/test split

numeric data
categorical data

encoding
scaling

scikit-learn

Pipeline

Logistic Regression

prediction

accuracy
F1

MLflow
experiment
run
parameter
metric
artifact/model

FastAPI

HTTP POST

```
request body
```

```
JSON
```

```
input validation
```

```
API response
```

Notice something important:

You are not learning a large number of new ideas today.

You are learning how ideas you already studied **connect**.

10. Thought Process — Raw Data → Tracked Model → API

Here is the thought process you should practice.

Step 1 — Understand the question

Ask:

```
What am I trying to predict?
```

Answer:

```
whether a customer purchases
```

Therefore:

```
target = purchased
```

Step 2 — Identify available information

Perhaps:

```
age  
income  
visits  
city
```

Those become possible features.

Step 3 — Inspect the data

Before modifying anything, ask:

How many rows?

Which columns?

Which types?

Any missing values?

Any duplicates?

What values does the target contain?

Step 4 — Clean only what needs cleaning

For example:

missing income → handle appropriately

duplicate row → remove if truly duplicate

wrong data type → convert

Avoid arbitrary transformations.

Step 5 — Separate X and y

Think:

X
=
information available before the prediction

y
=
answer we want the model to learn

Step 6 — Split before learning preprocessing

Create:

X_train
X_test
y_train
y_test

Then let the Pipeline learn preprocessing from the training data.

This helps prevent information from the test set leaking into training.

Step 7 — Decide preprocessing

Ask for every feature:

```
Is this numeric?  
Is this categorical?  
Is anything missing?  
Does scaling make sense?
```

Then create only the necessary preprocessing.

Step 8 — Build Pipeline

Think:

```
raw row  
  ↓  
preprocessor  
  ↓  
Logistic Regression
```

This single object becomes your main prediction workflow.

Step 9 — Start MLflow run

Now record your experiment.

Think:

```
What choices did I make?  
  ↓  
parameters  
  
How well did it perform?  
  ↓  
metrics
```

Step 10 — Train

```
pipeline.fit(...)
```

Now both:

```
preprocessing
+
model
```

learn what they need from the training data.

Step 11 — Evaluate

```
X_test
↓
pipeline.predict()
↓
predictions
↓
accuracy + F1
```

Then log those metrics.

Step 12 — Preserve the trained workflow

You need the fitted Pipeline later for prediction.

Conceptually:

```
trained Pipeline
=
fitted preprocessing
+
trained classifier
```

That is what your API should eventually use.

Step 13 — Build the prediction function

Do not put every detail directly into your FastAPI endpoint.

Think:

```
API endpoint
↓
prediction function
↓
Pipeline
```

This keeps responsibilities clearer.

Step 14 — Define API input

Specify the fields:

```
age
income
visits
city
```

and their expected types.

Step 15 — Call the model

```
validated input
  ↓
DataFrame / expected feature structure
  ↓
trained Pipeline
  ↓
prediction
```

Step 16 — Return JSON

Finally:

```
prediction
  ↓
Python dictionary
  ↓
JSON response
```

You have now completed the entire beginner pipeline.

11. Beginner-Friendly Pseudocode

Here is your complete project without the full implementation:

```
START PROGRAM
```

```
-----
TRAINING
-----
```

```
IMPORT required libraries
```

LOAD CSV using Pandas

DISPLAY:

- first rows
- shape
- columns
- data types
- missing values

IF necessary:

- fix simple data problems

DEFINE features X

DEFINE target y

SPLIT data into:

- X_train
- X_test
- y_train
- y_test

DEFINE numeric columns

DEFINE categorical columns

CREATE numeric preprocessing

CREATE categorical preprocessing

COMBINE preprocessing

CREATE Logistic Regression model

CREATE Pipeline:

- preprocessing
- ↓
- Logistic Regression

CREATE / SELECT MLflow experiment

START one MLflow run

LOG parameters:

- model name

- test size

- random state

- one important model parameter

TRAIN Pipeline using:

- X_train

- y_train

MAKE predictions using:

- X_test

CALCULATE:

- accuracy

- F1

LOG:

- accuracy

- F1

OPTIONALLY:

- log trained Pipeline/model

OR:

- save and log one simple artifact

END MLflow run

SAVE or otherwise make trained Pipeline
available to prediction application

```

-----
PREDICTION FUNCTION
-----

DEFINE prediction function:

    RECEIVE:
        age
        income
        visits
        city

    BUILD one-row input

    SEND input to trained Pipeline

    GET prediction

    RETURN prediction


-----
FASTAPI
-----

CREATE FastAPI application

DEFINE expected request structure:

    age = numeric
    income = numeric
    visits = numeric
    city = text

CREATE POST /predict endpoint

WHEN request arrives:

    VALIDATE input

    CALL prediction function

    RETURN:

    {

```

```
        "prediction": result
    }

END
```

If you can understand every box in this pseudocode, you understand the project's architecture.

12. Suggested Solving Approach

Use four small stages instead of trying to build everything at once.

Stage 1 — Pandas

First get:

```
CSV
↓
DataFrame
↓
clean data
```

Stop there until it works.

Stage 2 — scikit-learn

Then get:

```
DataFrame
↓
features + target
↓
train/test
↓
Pipeline
↓
train
↓
predict
↓
evaluate
```

Do not add FastAPI yet.

Stage 3 — MLflow

After training works:

```
training experiment
  ↓
MLflow run
  ↓
parameters + metrics
```

Confirm that your run contains the details you intended to track.

Stage 4 — FastAPI

Only then connect:

```
new input
  ↓
FastAPI
  ↓
prediction function
  ↓
trained Pipeline
  ↓
JSON response
```

This debugging order is much easier than trying to create everything simultaneously.

13. Responsibilities of Each Tool

This distinction is one of the most important parts of Day 120 Part-2.

Pandas → Data Handling

Use Pandas for:

```
loading CSV
inspecting rows
checking columns
checking missing data
simple cleaning
constructing DataFrames
```

Mental model:

```
Pandas
=
work with tabular data
```

scikit-learn → Machine Learning

Use scikit-learn for:

```
train/test split
preprocessing
encoding
scaling
Pipeline
model training
prediction
evaluation
```

Mental model:

```
scikit-learn
=
build and use the ML workflow
```

MLflow → Experiment Tracking

Use MLflow for:

```
experiment
run
parameters
metrics
tags
artifacts
trained model logging
comparison/history
```

Mental model:

```
MLflow
=
record what happened during ML training
```

It does **not** replace scikit-learn.

FastAPI → Application Interface

Use FastAPI for:

```
HTTP endpoint

request validation

receiving JSON

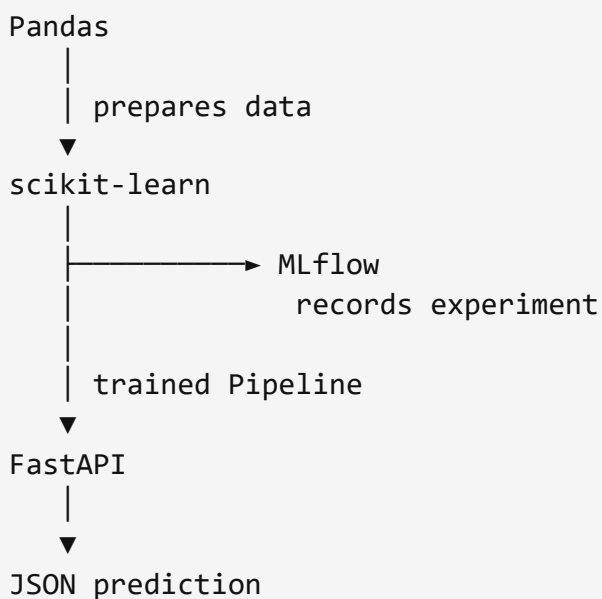
calling prediction logic
```

```
returning JSON
```

Mental model:

```
FastAPI
=
allow another application/client
to ask your Python model for a prediction
```

Together



Or even simpler:

Tool	Question it answers
Pandas	How do I load and inspect my data?
scikit-learn	How do I preprocess, train, evaluate and predict?
MLflow	What happened during this experiment?
FastAPI	How can another program request a prediction?

14. Easy Edge Cases

Edge Case 1 — Missing dataset value

Suppose:

```
age = 30
income = missing
visits = 4
```

Do not blindly train.

Think:

```
Should this row be removed?  
  
Should the missing value be filled?  
  
What does a missing income actually mean?
```

For this project, choose one simple and reasonable approach.

Edge Case 2 — Missing API input

Expected:

```
{  
  "age": 30,  
  "income": 50000,  
  "visits": 4,  
  "city": "Pune"  
}
```

But the user sends:

```
{  
  "age": 30,  
  "city": "Pune"  
}
```

Important information is missing.

The request schema should describe required inputs so the API can reject invalid requests rather than sending incomplete data into the model.

Edge Case 3 — Invalid Numeric Input

Bad request concept:

```
{  
  "age": "thirty",  
  "income": 50000,  
  "visits": 4,  
  "city": "Pune"  
}
```

age should be numeric.

Your input model should expect the correct type.

Edge Case 4 — Unexpected Category

Training data contained:

```
Bengaluru  
Mumbai  
Delhi  
Pune
```

But your API receives:

```
Chennai
```

An encoder may need an appropriate strategy for unseen categories.

For example, at a beginner level you can investigate the idea of configuring one-hot encoding to safely handle an unknown category rather than unexpectedly crashing.

The important lesson is:

|

Think about what happens when prediction data differs from training data.

Edge Case 5 — Very Small Train/Test Split

Suppose your whole dataset contains:

```
10 rows
```

An 80/20 split gives only:

```
8 training rows  
2 testing rows
```

Metrics calculated from two test examples are unstable and not very informative.

For today's learning project that is okay as a demonstration, but you should recognize the limitation.

Edge Case 6 — Model Cannot Make a Prediction

Possible causes include:

```
missing model  
wrong feature names  
wrong data types  
unexpected preprocessing problem
```

```
wrong number of features
corrupted saved file
```

Do not automatically assume FastAPI is the problem.

Trace the flow:

```
request
  ↓
input conversion
  ↓
Pipeline
  ↓
prediction
```

and identify where the failure occurs.

Edge Case 7 — Different Preprocessing at Prediction Time

Training:

```
StandardScaler
OneHotEncoder
LogisticRegression
```

Prediction:

```
raw values
  ↓
LogisticRegression directly
```

This is wrong.

The model was trained on transformed features.

Correct conceptual flow:

```
new raw values
  ↓
same fitted Pipeline
  ↓
prediction
```

Edge Case 8 — Forgetting MLflow Information

Imagine the run says:

```
accuracy = 0.85
```

```
F1 = 0.82
```

but you forgot what model produced it.

That experiment is much less useful.

Log important context such as:

```
model_name
test_size
random_state
important model parameter
```

Similarly, logging parameters without the evaluation metrics leaves you unable to tell how the run performed.

15. Common Mistakes to Avoid

1. **Training before inspecting the dataset.**
2. **Including the target column inside X.**

That can cause target leakage.

3. **Cleaning the test data using information learned from the whole dataset.**
4. **Fitting the scaler separately on test data.**
5. **Encoding categories differently during training and API prediction.**
6. **Training preprocessing outside the Pipeline and forgetting about it later.**
7. **Using a model simply because it sounds advanced.**

Logistic Regression is completely appropriate for a beginner binary-classification capstone.

8. **Evaluating on training data and calling that final performance.**
9. **Looking only at accuracy when another metric such as F1 may provide useful information.**
10. **Logging a metric to MLflow without logging the configuration that produced it.**
11. **Creating many MLflow runs just to make the project seem sophisticated.**

One clear run is enough.

12. **Putting model-training code directly inside /predict.**

The model should not normally retrain every time someone requests a prediction.

13. **Loading and cleaning the entire training dataset on every prediction request unnecessarily.**
14. **Creating a new scaler or encoder for every API request.**
15. **Sending JSON directly into a model without recreating the feature structure it expects.**

16. **Returning NumPy-specific objects that cannot easily become JSON instead of converting the result to an ordinary Python value where necessary.**
17. **Adding databases, Docker, cloud deployment or authentication before the basic local workflow works.**
18. **Thinking successful API execution proves the model is good.**

API correctness and model quality are different issues.

16. Five Quick Self-Check Questions

Question 1 — Data

Given:

```
age
income
visits
city
purchased
```

and the goal is predicting purchased:

Which columns should belong to:

```
x
```

and which should belong to:

```
y
```

?

Question 2 — Pipeline

Why is this:

```
preprocessing
  ↓
Logistic Regression
```

inside one Pipeline generally safer than manually preprocessing training data and then trying to remember those transformations later when your API receives data?

Question 3 — MLflow

Which of these should normally be logged as a **parameter**, and which as a **metric**?

```
C = 1.0  
  
test_size = 0.20  
  
accuracy = 0.86  
  
F1 = 0.82
```

And why would knowing only:

```
F1 = 0.82
```

be insufficient when reviewing the experiment later?

Question 4 — FastAPI

Suppose /predict receives:

```
{  
  "age": 29,  
  "income": 48000,  
  "visits": 4,  
  "city": "Delhi"  
}
```

What should happen between receiving this JSON and returning:

```
{  
  "prediction": 1  
}
```

?

Try to describe the complete flow without code.

Question 5 — Complete Workflow

Put these in the correct conceptual order:

```
MLflow metric logging  
  
JSON response  
  
train/test split  
  
Pandas loading  
  
FastAPI endpoint
```

```
Pipeline training  
  
feature/target separation  
  
prediction  
  
data cleaning  
  
model evaluation
```

If you can correctly explain the sequence and the purpose of each step, you understand the core Day 120 Part-2 workflow.

17. Hint Only

Build this final project one layer at a time.

Your first checkpoint is:

```
CSV  
↓  
Pandas  
↓  
clean DataFrame
```

Then:

```
clean DataFrame  
↓  
X + y  
↓  
train/test split
```

Then:

```
X_train  
↓  
preprocessing  
↓  
Logistic Regression
```

Put those last two steps together:

```
Pipeline  
|  
├─ preprocessing  
└─ Logistic Regression
```

Then ask:

```
Can my Pipeline train?  
  
Can it predict X_test?  
  
Can I calculate accuracy and F1?
```

Only after those work, add:

```
MLflow experiment  
    ↓  
one run  
    ↓  
parameters  
    ↓  
metrics  
    ↓  
optional model/artifact
```

Finally:

```
new customer JSON  
    ↓  
POST /predict  
    ↓  
FastAPI  
    ↓  
prediction function  
    ↓  
same trained Pipeline  
    ↓  
prediction  
    ↓  
JSON
```

Your final Day 120 Part-2 checklist is therefore:

```
[ ] I can load data with Pandas  
  
[ ] I can inspect and clean simple data  
  
[ ] I understand features and target  
  
[ ] I can create train/test data  
  
[ ] I understand why preprocessing is needed  
  
[ ] I can keep preprocessing and the model
```

together using Pipeline

- [] I can train one supervised model
- [] I can evaluate it
- [] I understand MLflow experiment vs run
- [] I know parameter vs metric
- [] I can track one meaningful training run
- [] I understand why the fitted Pipeline must be preserved for future predictions
- [] I can create a prediction function
- [] I understand what a FastAPI endpoint does
- [] I can describe a JSON request and response
- [] I understand the complete flow:
 - data
 - ↓
 - preprocessing
 - ↓
 - model
 - ↓
 - evaluation
 - ↓
 - MLflow
 - ↓
 - prediction
 - ↓
 - FastAPI
 - ↓
 - JSON

That is enough for the final beginner project. The success criterion is **not** unusually high model accuracy or sophisticated infrastructure. It is being able to explain why every stage exists and how the output of one stage becomes the input to the next.

By the way, ChatGPT Images 2.5 can turn a rough idea into a finished image, with richer textures and details you can refine. Want me to create an image of a Day 120 Part-2 ML workflow study poster?